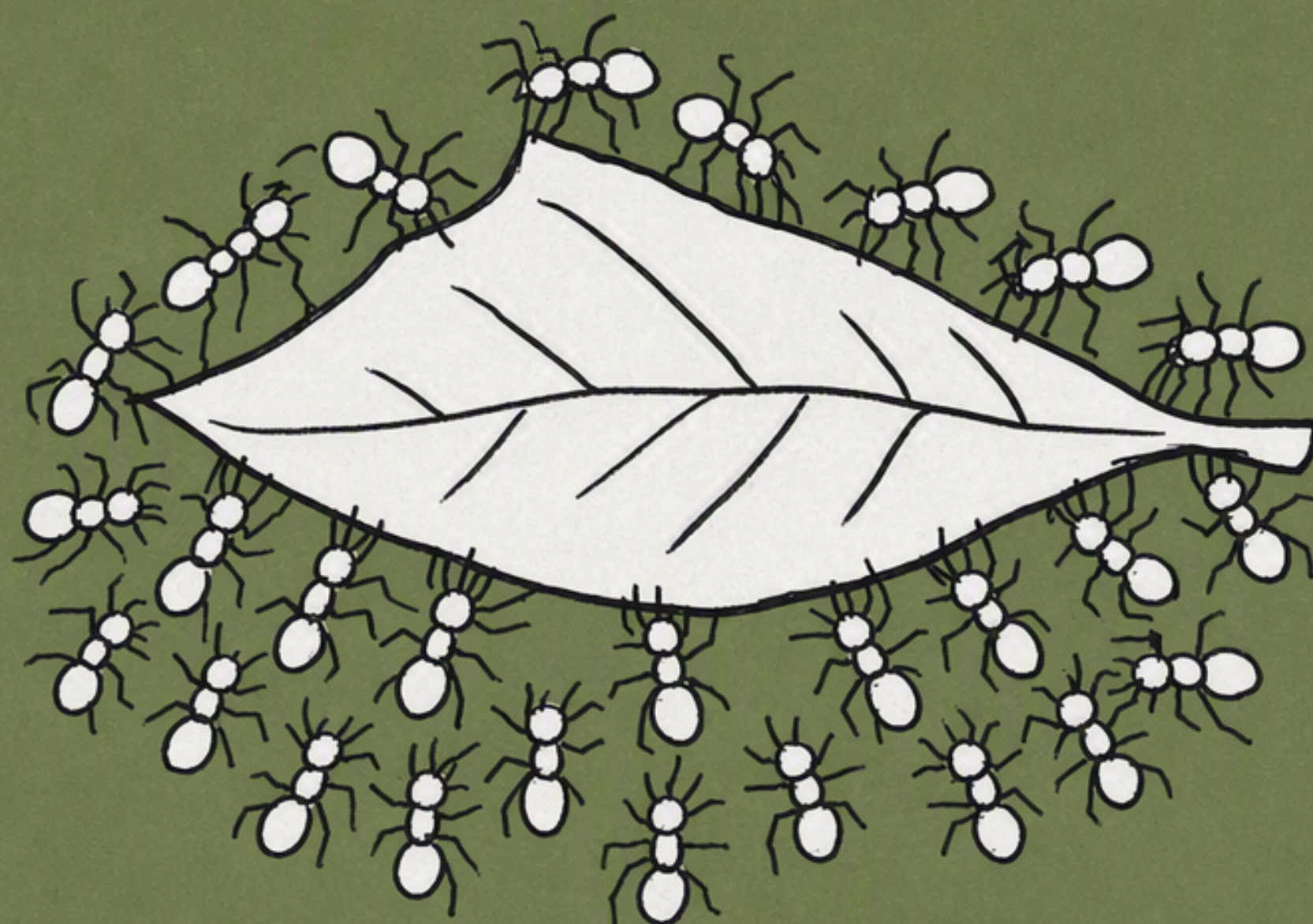




💡 Multi-Agent · 工程实践 📅 2026.8.4 📖 1.4 万字 ⌚ 约 28 分钟

Multi-agent 的新趋势：从 Agent Team 到 Agent Swarm（下）

上篇讲到，Anthropic 和 Codex 的官方文档，都把"多 agent 改同一份文件"标成了不适用场景。这篇从硬闯这条线的 Cursor 开始。

第二章（续）· agent 之间怎么协同

● Cursor-上千个 agent 改同一个代码仓库

在我过往的认知里，多 agent 适合的是并行调研这类任务：同时查 50 个 benchmark、同时分析 500 个作者的文风。结构相同、互相独立、单个不难，天然适合 map-reduce。

但写作和代码生成这类活，我一直觉得不合适。理由很直接：你让几个 agent 去改同一个代码仓库，冲突是必然的。

人类团队靠"一个人负责一个模块"、"改之前先说一声"和"资深的工程师负责解决冲突"。但几百个 agent 同时动手、还谁都不跟谁说话，就很难办了，所以那会儿的结论是"这类任务还是单 agent 老老实实做"。

可基模越来越强，而想上多 agent 的理由又很朴素：**我们就是要快啊。** 10 分钟并行写完一个代码仓库，和 60 分钟串行写完，这中间的差别不是效率的百分比，是能不能改变工作方式。

所以问题就变成了：那冲突到底怎么解决？

Cursor 今年发的两篇博客有做这方面的尝试。

• 架构是迭代出来的，不是设计出来的

《Towards self-driving codebases》(2026.2.5) 讲他们怎么让大量 agent 协作开发一个真实项目：

- 累计编排了好几千个 agent。不过要注意另一个数：单次运行同时在线的峰值是几百个，原文说大多数 run 都在几百个 agent 这个量级上封顶。
- 峰值约 1,000 commits/小时，连跑一周，累计约 1,000 万次工具调用，而且"一旦启动就不需要我们任何干预"。

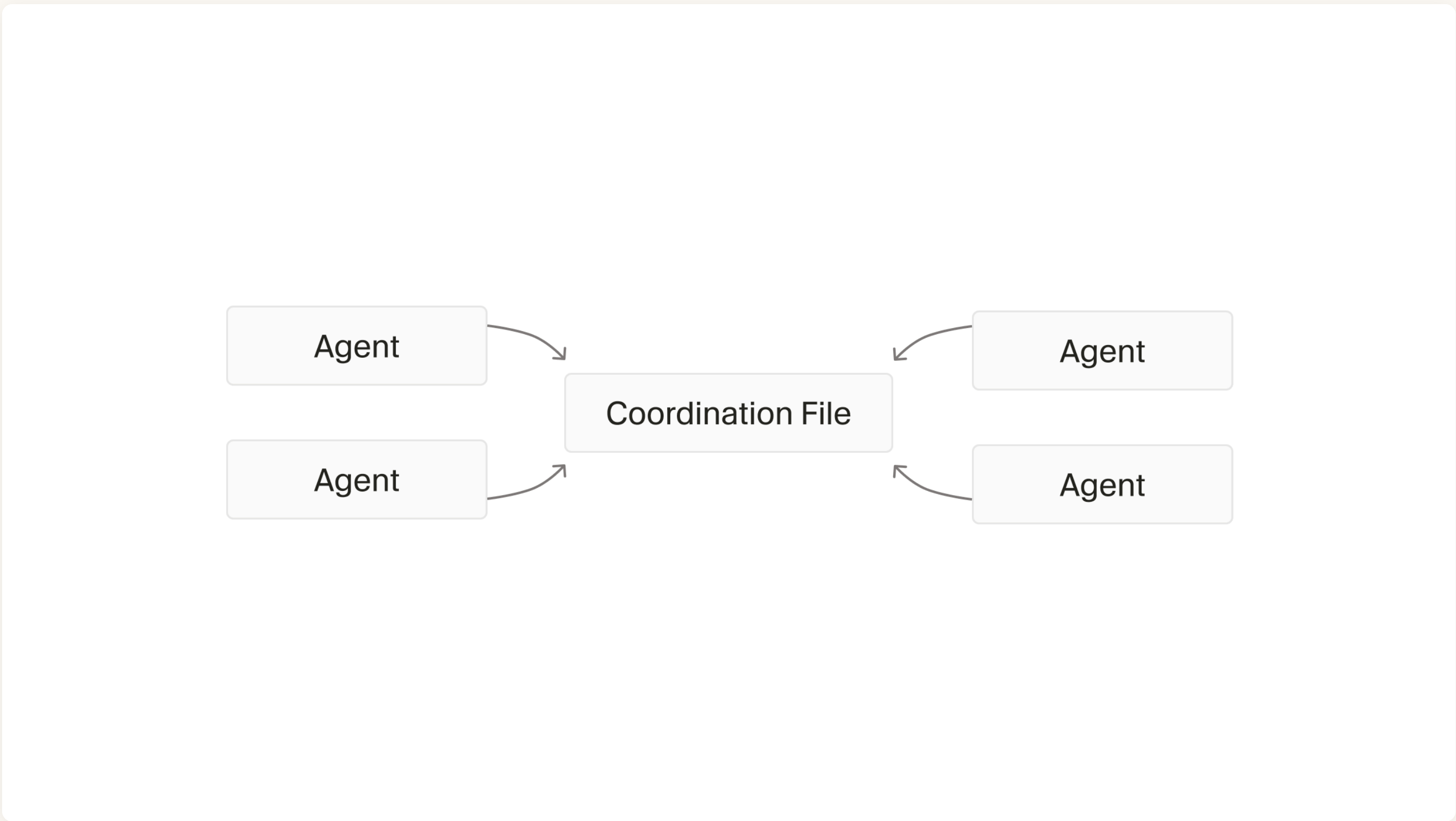
架构是迭代出来的，不是设计出来的，这个演进过程比最终形态更有信息量。

往下看之前，先跟大家把几个角色名说清楚——后面四版翻来覆去，就是在调整它们的关系：

- planner (规划者)：拿着目标，决定"这事该拆成哪几件"，自己不写代码。
- worker (干活的)：领一件具体的活，把它做完，不关心别人在干什么。
- executor (执行主管)：介于两者之间，负责盯着计划落地、派活、验收。
- judge (验收员)：干完之后单独跑一遍，判断"这算做完了吗"。

套项目组来理解就行：planner 是只出方案不动手的人，worker 是照着工单干活的人，executor 是项目经理，judge 是 QA。Cursor 迭代了四版，就是这几个岗位怎么被增、被删、被合并。

第一版：一堆平级 agent + 一个共享状态文件加锁。



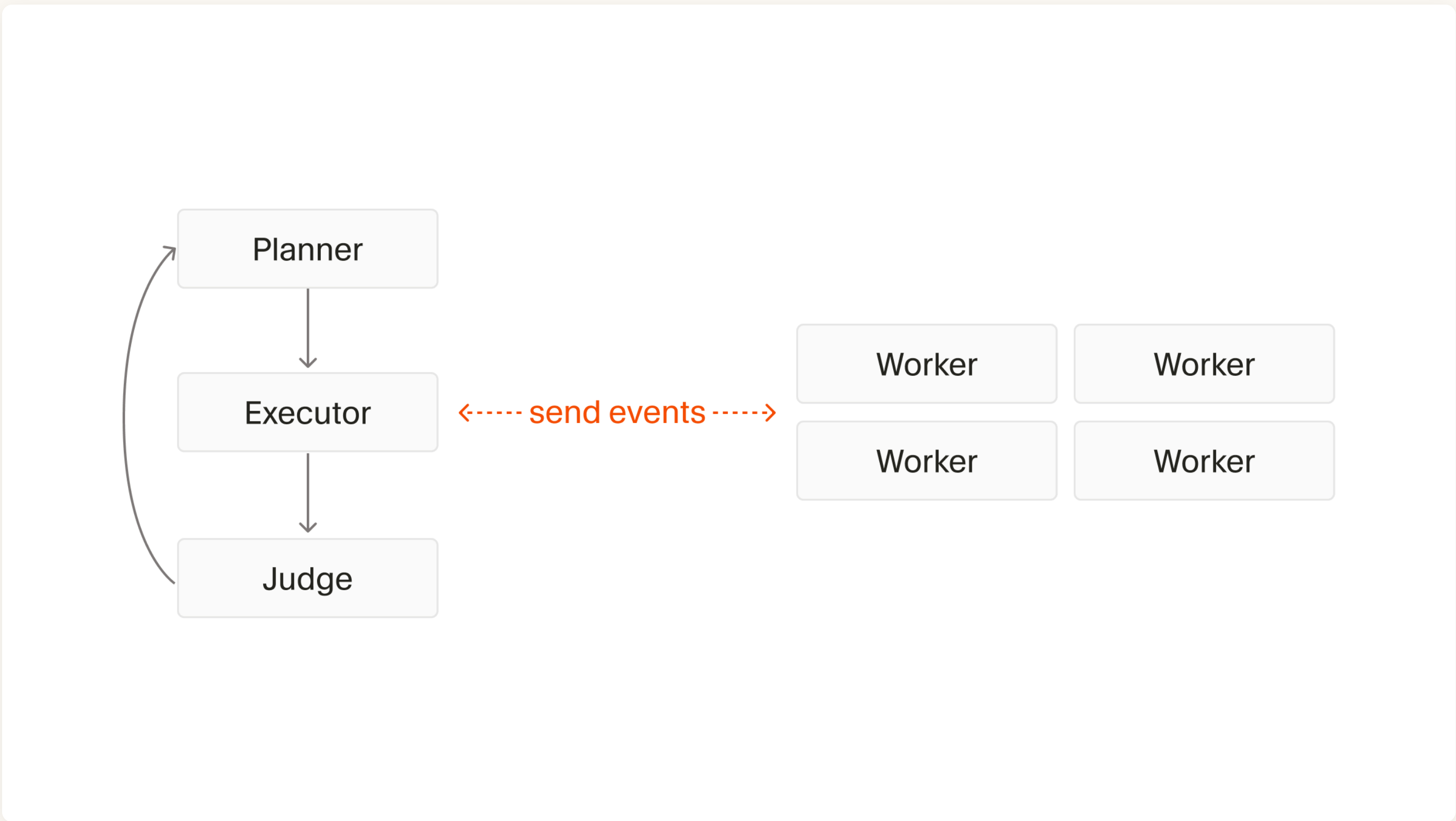
四个地位相同的 agent，全都去读写中间那一个 Coordination File

思路很直觉：既然大家要协调，那就搞一个共享文件写清楚"谁在干什么"，加锁防止互相踩。

问题一，模型不懂锁。原文说 agent 会把锁攥太久、会忘了释放、会在不该加锁解锁的时候乱来——它们压根不理解"持有一把锁"这件事意味着什么。结果是 20 个 agent 的吞吐量掉到 1—3 个的水平，绝大部分时间都耗在等锁上。

问题二：没有结构的时候，没有一个 agent 愿意接大活。它们全都在规避冲突，专挑小的、安全的改动做，谁也不为整个项目负责。这个副作用不是性能问题，是责任问题——你以为你有 20 个开发，实际上你有 20 个只肯改错别字的人。

第二版：引入 planner / executor / worker / judge 四个角色。



Planner → Executor → Judge 串成一个闭环，judge 判完再转回 planner；右边那一排 worker 跟 executor 之间收发事件

既然平级不行，那就上角色。planner 先把做法和交付物定下来，交给 executor 当唯一的主事人，executor 再派 worker 去干，最后由一个独立的 judge 在 executor 收工后跑一遍验收。

这一版好了不少，但撞上两个问题。

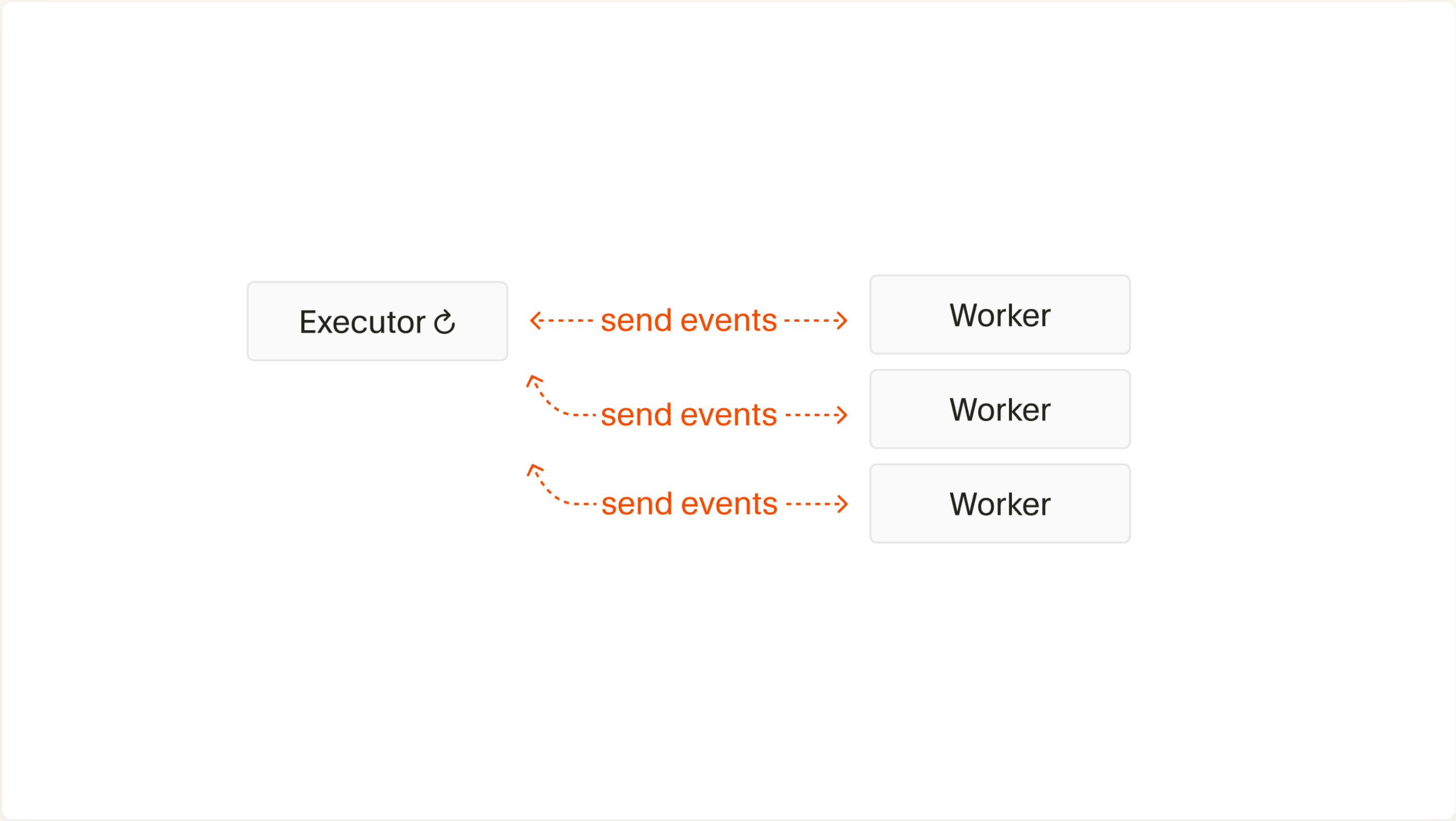
一是整个系统被最慢的那个 worker 卡住。原文的评价很直接：太僵硬了。

二是所有规划都在最前面一次性做完，等真跑起来发现新情况的时候，系统很难再动态调整——而
在一个真实项目里，"跑着跑着发现事情不是那样"几乎是常态。

顺带说两个被砍掉的角色，理由都很实在：

- judge 被砍，是因为他们发现 agent 照着指令干到底这件事做得还不错，那就为了让系统简单点，去掉。
- 一个集中的 integrator 也被砍了，原因是它立刻成了明显的瓶颈——它当初被加进来，恰恰是为了做全局质量把关、顺便消化"太多 worker 同时推代码、解冲突、合并"的争抢。结果它立刻成了明显的瓶颈：几百个 worker，却只有一道所有产出都必须经过的关口，原文的评价是这就成了"繁文缛节"（red tape）。

第三版：砍掉独立的 planner，让 executor 兼做规划。



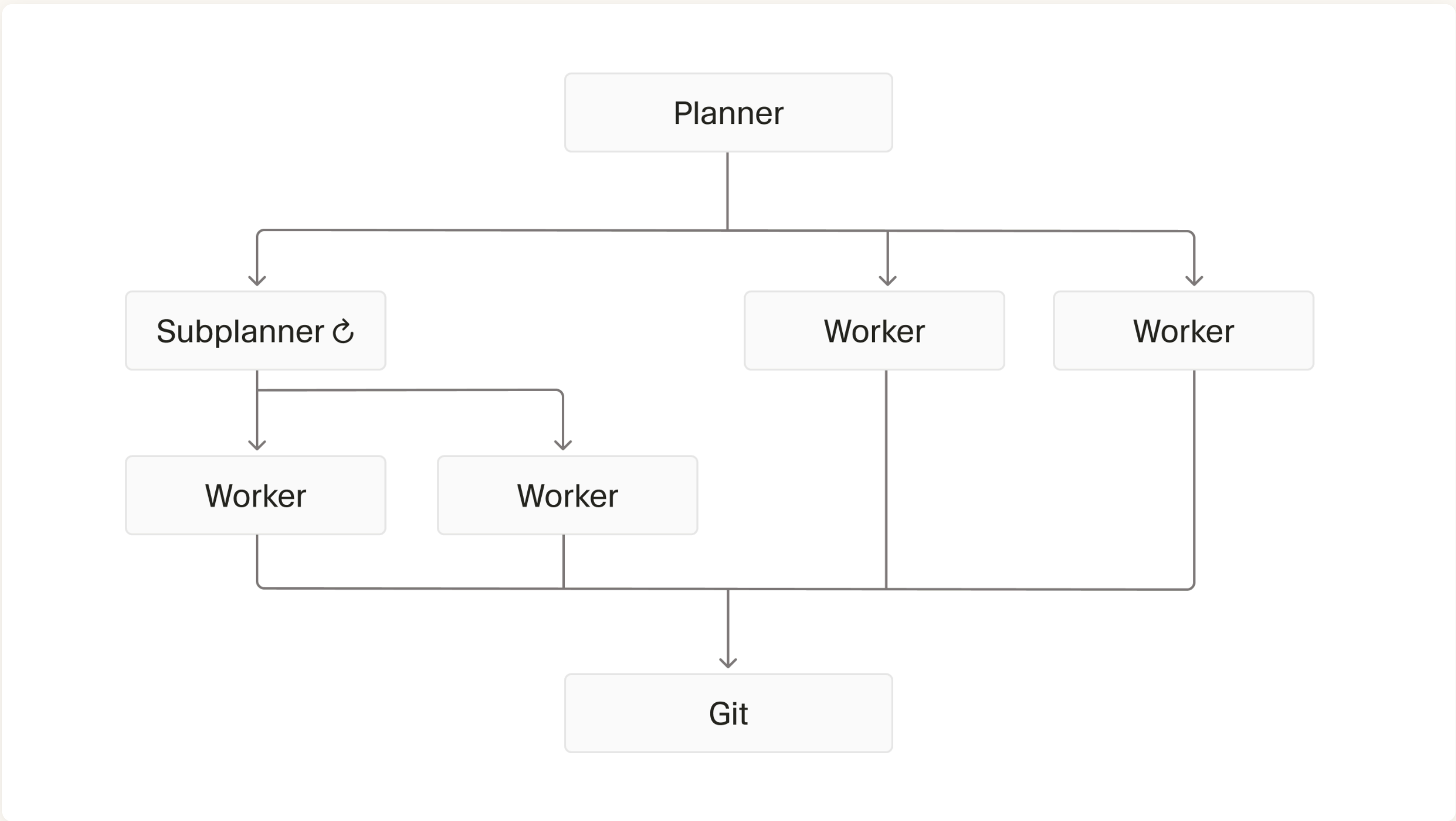
只剩一个带循环箭头的 Executor，直接跟每个 worker 收发事件——planner 和 judge 都没了

既然一次性规划太僵硬，那就让 executor 边干边规划。

结果 executor 开始出现各种病理行为：随机睡觉、把正在跑的 agent 停掉、自己动手干活、死活不肯多派任务（只肯派那么几个窄得不能再窄的）、不好好合并 worker 的改动、还没做完就宣布完成。

原因是它一个人被塞了太多角色和目标。

最终版：递归的 planner。



最终形态：一个 root planner 拥有全部 scope、自己不写代码，觉得可以再拆就派生 subplanner，递归下去；worker 不知道更大的系统存在、不跟任何人通信、在自己那份仓库副本上干活，做完写一份 handoff。所有产出统一落到 Git

最终形态是把"分层"这件事做成了递归：一个 root planner 拿着全部 scope，自己不写一行代码，觉得某块还能再拆就派生一个 subplanner，subplanner 再往下拆——planner 这一层可以长成任意深的树。树的最末端才是 worker：worker 不知道更大的系统存在，不跟任何人通信，干完写一份 handoff（交接说明）就走人。所有产出统一落到 Git。

这里有个机制值得单独说，因为它是后面所有冲突故事的前提：每个 worker 先给自己拷一份完整的代码副本，在副本里改，不直接动主仓库。好处显而易见——你改你的、我改我的，改的过程中谁也碰不到谁，几百个人可以真正同时动手。但代价是，冲突并没有消失，只是被推迟到了"合回主干"的那一刻。

还有一条反直觉的原则值得单独拎出来：要显式地为吞吐量做设计，而不是为 100% 的正确性做设计。

他们试过"每一次提交前都必须完全正确"，结果是整个系统被串行化：只要有人手滑改了个接口、打错一个字，几百个 agent 就一起停在那儿等修好。改成允许一点松弛之后反而更快——因为每个 agent 都可以信任"这个毛病会有同事很快修掉"。代价是要专门留一条常绿分支，让一个 agent 定期打快照、在发布前统一收一遍尾。

• 冲突后来是怎么压下去的

但冲突（两个agent同时在改同一份代码）还是会存在，[《Agent swarms and the new model economics》](#)（2026.7.20）第二篇提到：

失效模式一：脑裂（split-brain）。两个互不知情的 planner，在代码库的不同位置，用两套不一样的做法把同一个概念各实现了一遍。这不是"白干一遍活"那么简单——是这个仓库里从此并存两种互相打架的设计，而且谁都不知道对方存在。

解法是提示词层面的，两条：

1. 决策不许下放。 planner 自己把设计决定拍下来，而不是把"这块你自己看着办"派下去。道理很直白——一旦把"决定"派下去，下面两个人各拍各的，就一定不一样。

2. 拆活的时候保证不重叠。要求 planner 在往下分子树的时候，确保任意两棵子树不会去决定同一个问题。

归拢起来就一条：把"做决定"和"干活"分开——决定必须在能看到全局的那一层做完，往下派的只能是实现，不能是决策。

失效模式二：planner 之间抢文件。这个更难，因为两个 planner 知道彼此存在，却还是在同一批文件上来回改。原文：问题在于这里有两份互相矛盾的"现实图景"，而合并工具是修不了分歧的。合并工具能解决"两个人改了同一行"，解决不了"两个人对这件事的看法不一样"。

举个例子（我编的，原文没给）：planner A 定了内部字符串统一用 UTF-8，planner B 定了用 UTF-16，各自往下派活。两摊代码各自都能编译、测试都过，合并时 git 一声不吭——因为他们改的压根是不同的文件，git 只比对行，不看意思。等到 A 的函数把结果传给 B 的函数，才炸。

解法是三步：

1. agent 把设计决策写进一份共享设计文档——不是塞在代码注释里，是单独一份 doc。
2. 依赖这个决策的代码，带一个能被编译器校验、并且能追溯回那份文档的引用。
3. 两个 planner 无意中互相矛盾时，一个 reconciler 去把文档合并掉，然后这些引用把结论传播到下游。

还是上面那个例子：A 把"用 UTF-8"写进一份决策文档，所有相关代码挂一个指回它的引用；B 写了矛盾的决策之后，reconciler 裁定 UTF-8 胜出、改掉文档——于是 B 那摊代码当场全部编译失败，它手下的 worker 撞上报错，顺着引用找到文档，跟着改。

第 2 步是整套机制的关键，但原文没说这个引用具体长什么样（是个常量？类型？注解？），只说了它"可被编译校验"。不过同一篇文章里另一处讲"僵化"问题时，用的是同一个思路，那段写得就具体多了——

他们发现 agent 有个毛病：在有人类参与的代码库里待久了，学会了"核心代码碰不得"，哪怕它确实该改。于是他们允许 agent 故意把编译搞坏：某个 agent 如果判断这处核心改动值得做，就可以越出自己的范围打一个补丁，并留一段注释说明为什么。然后编译器会把这个改动带到整个系统，所有依赖旧设计的地方全部构建失败；每一个撞上这些报错的 agent，都会顺着找到那段注释、读到理由，再把自己那块改成匹配的。

这就是"用编译器当广播机制"。核心代码一改，所有依赖旧设计的地方立刻编译失败，谁都躲不过去——光留一段注释是做不到这件事的，因为没人会主动去读一段跟自己无关的注释。这里的分工是：编译错误负责把人拦下来，注释负责告诉他为什么。

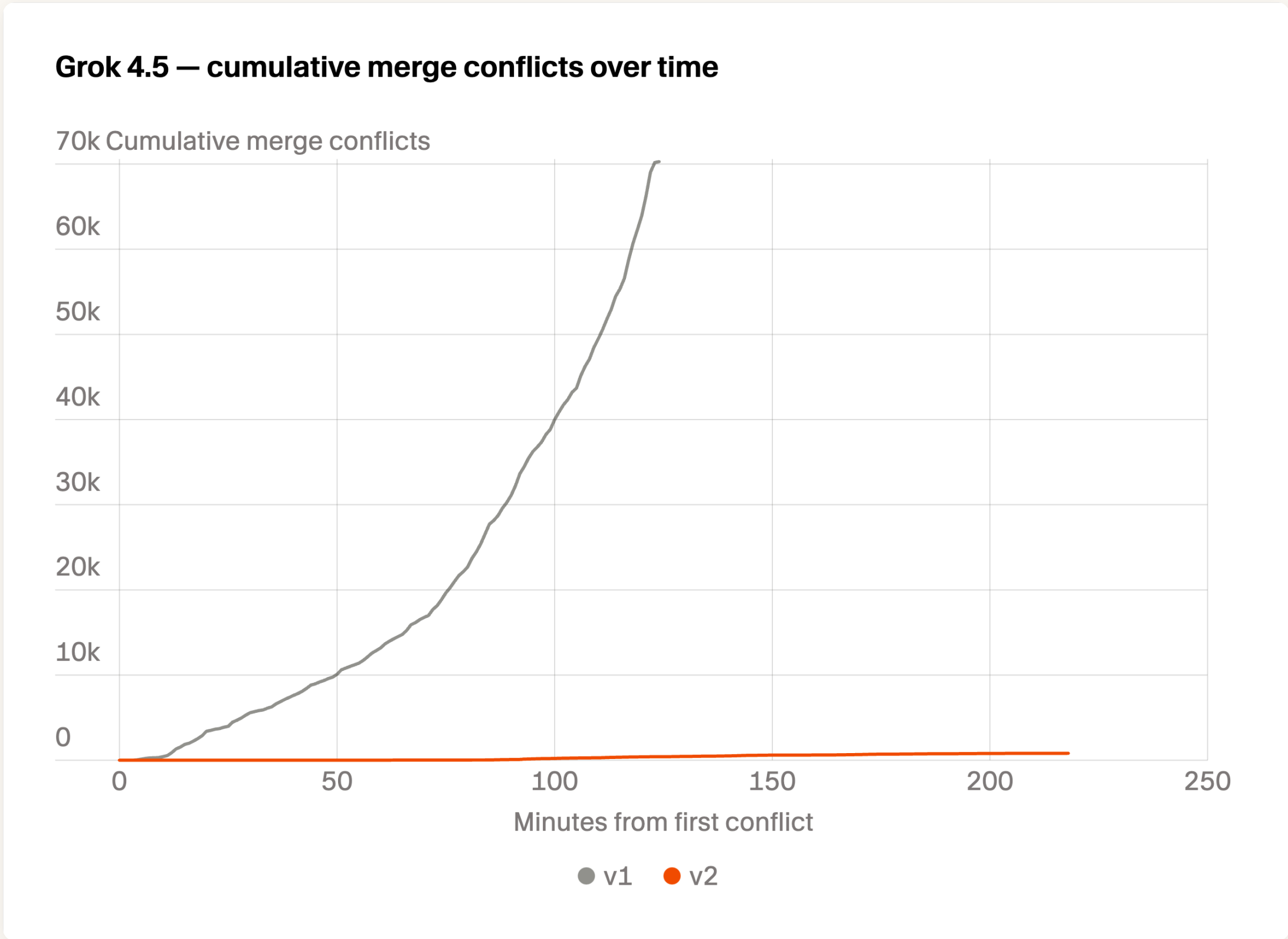
失效模式三：合并冲突爆炸。旧系统在被暂停之前（不到两小时）就累计了 7 万多次冲突，而且还在加速；最热的那一个文件吃掉 7,771 次冲突，被 1,173 个不同的 agent 碰过。

解法方法是：

- 一个中立的第三方仲裁 agent：冲突发生时它介入，代表所有相关方把冲突解掉。原文说它唯一的目标就是公正和高效，类似工程团队里的合并队列。关键在"中立"——它不是任何一方的 planner，所以不会偏袒。

- 自动拆分过大的文件：给 worker 一个"举报臃肿文件"的口子。一旦某个文件被举报，先冻结它的新提交，然后派一个外部 agent 把它拆成小模块。原文管这些膨胀失控的文件叫"巨型文件"，评价是：这些巨型文件会把一切都噎住。

这两件事之后，完整 4 小时的冲突降到 1,000 次以下，全库最热的文件只有 47 次。



同一个 Grok 4.5，v1（灰）在两小时内累计冲突冲到 7 万还在加速，v2（橙）跑满 200 多分钟几乎贴着 0

还有个数字特别能说明"忙"和"有产出"是两回事：旧 run 头两个小时产出了 68,000 次 commit，是新 run 节奏的 70 倍。一种读法是它更高产，另一种读法是——那些 commit 大部分是 thrash、争抢和空转。包结构上也有物证：旧 run 摊到了 54 个 crate，其中有三个各自独立的 SQL 包；新 run 早早收敛在 9 个，之后再没加过。

还有一个我很喜欢的设计叫 Field Guide：一个完全由 agent 自己拥有的文件夹，`index.md` 会自动注入每个 agent 的开头，唯一的约束是行数预算。理由是：模型权重是冻结的，所以真正值得记下来的，恰恰是那些"没想到"的遭遇——这样下一个 agent 的路径就能短一点。

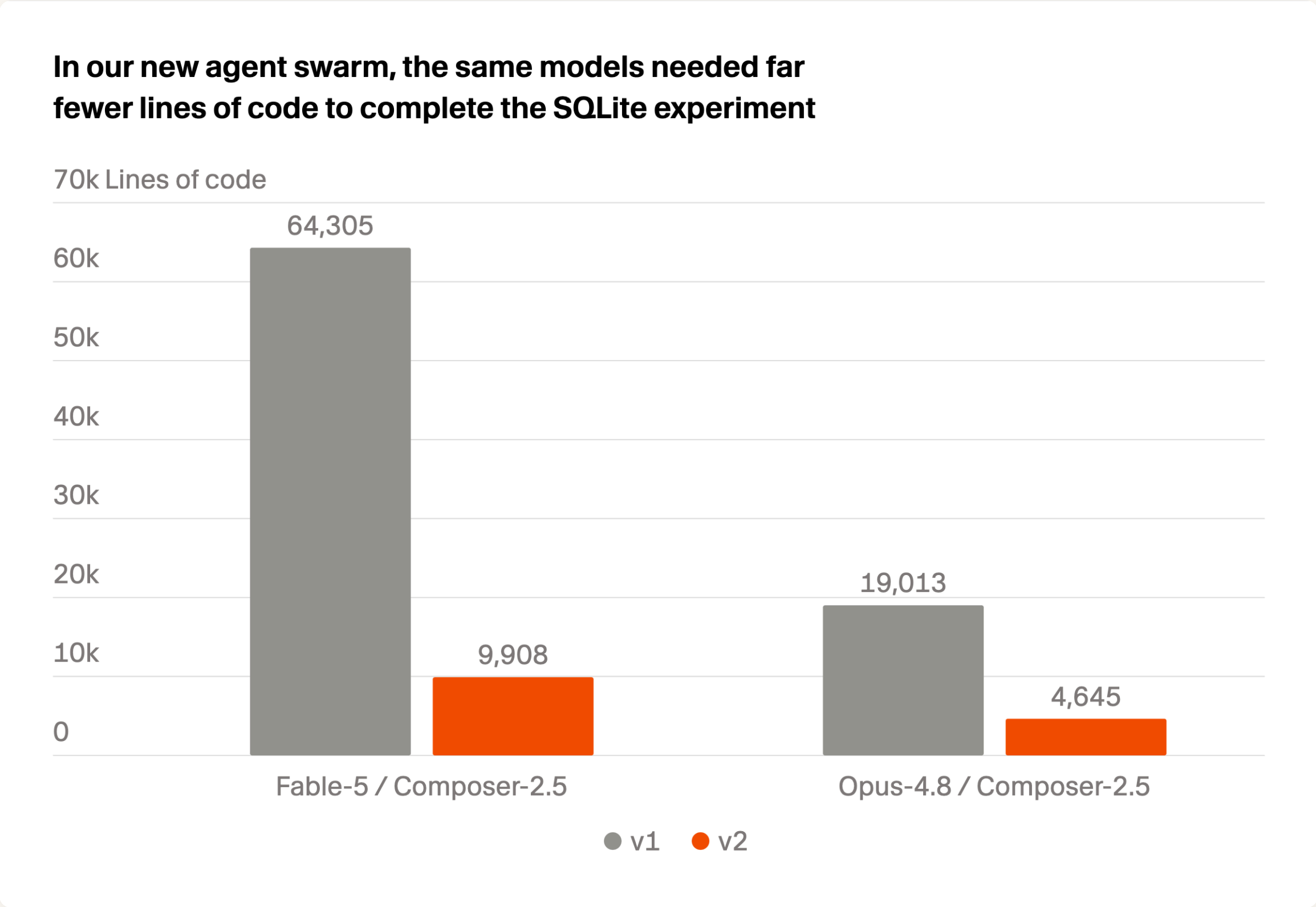
• SQLite 实验：只给一份 835 页手册

实验设置排得很干净：只给一份 835 页的 SQLite 手册，不给源码、不给测试套件、不给二进制、不给联网，让 swarm 用 Rust 把手册的全部内容实现出来。评分用 sqllogictest（百万级已知答案的 query），而且swarm 从头到尾不知道这个测试套件存在；每次跑完他们还人工复查代码和运行过程，专门查有没有作弊和走捷径。

结果：4 小时截点上，新框架 73—85% 通过率，旧框架 11—77%（其中 Grok 4.5 的旧框架 run 在两小时之前就被暂停了）。

但这里有句比截点更重要的话：新框架的每一种配置，最后都跑到了 100% 通过。

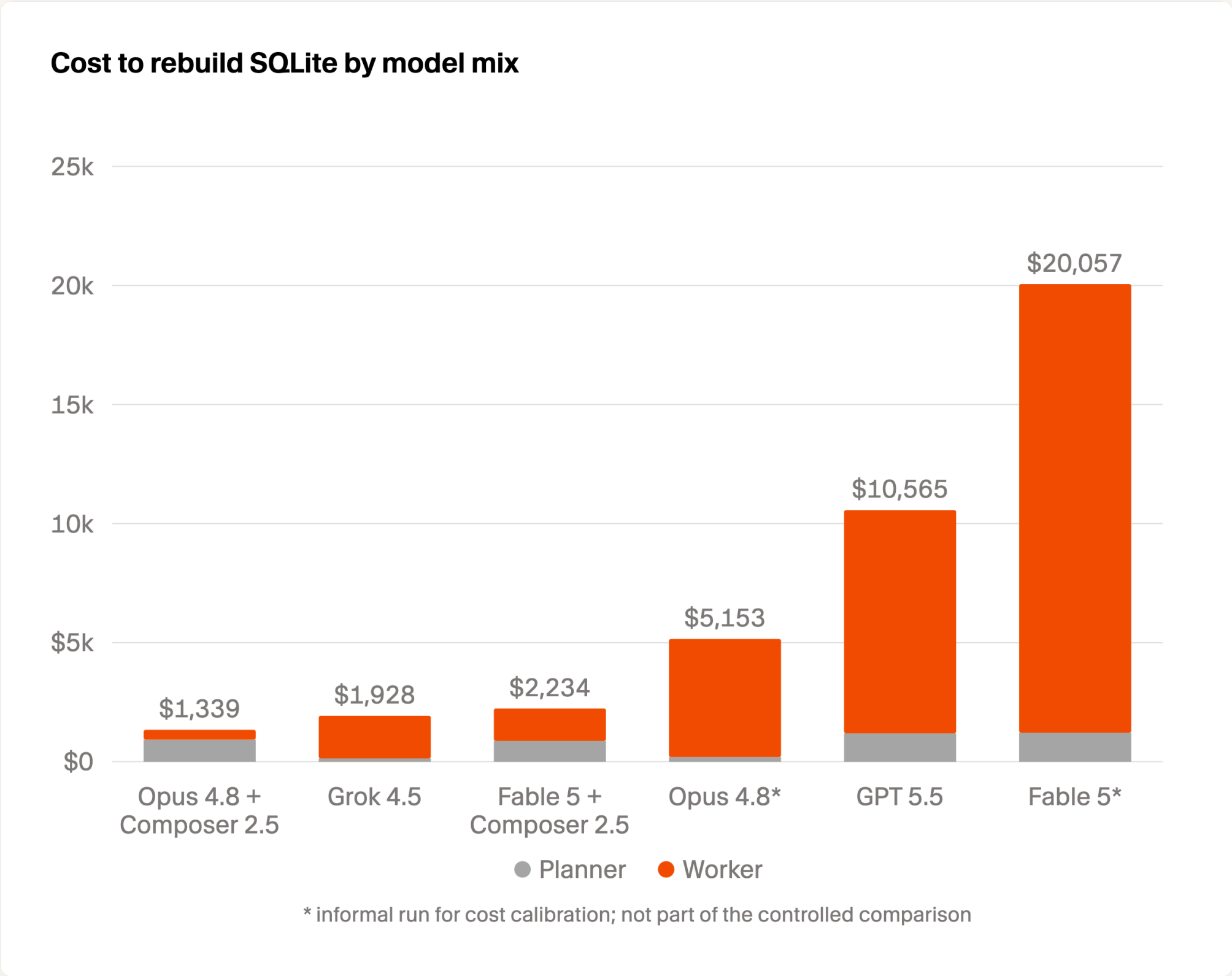
代码量的对比更能说明问题：



同样的模型、同样的任务，Fable 5 组合从 64,305 行降到 9,908 行；Opus 4.8 组合从 19,013 行（97% 通过）降到 4,645 行（100% 通过）

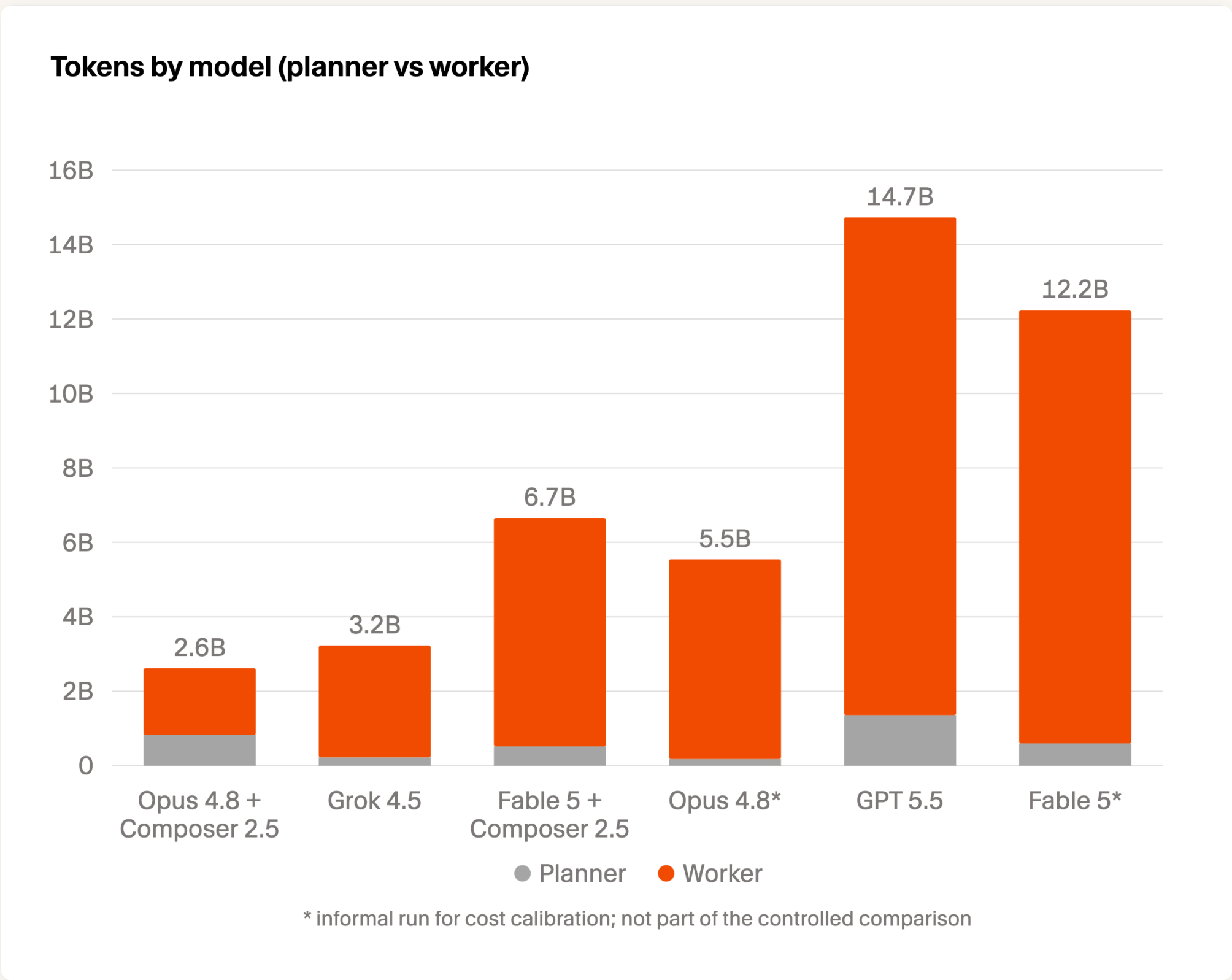
少写四分之三到六分之五的代码，把同一件事做得更对——这比通过率涨几个点更能说明组织方式的价值。

• 钱到底花在哪



重建 SQLite 的成本按模型组合拆分，灰色是 planner、橙色是 worker

区间是 \$1,339（Opus 4.8 混合配置）到 \$10,565（GPT-5.5 全包），质量接近，成本差 8 倍。拆开看更夸张：GPT-5.5 全包那次，光 worker 就花了 \$9,373；而 Opus 4.8 做 planner、Composer 2.5 做 worker 那次，整个 worker 集群总共只花了 \$411。



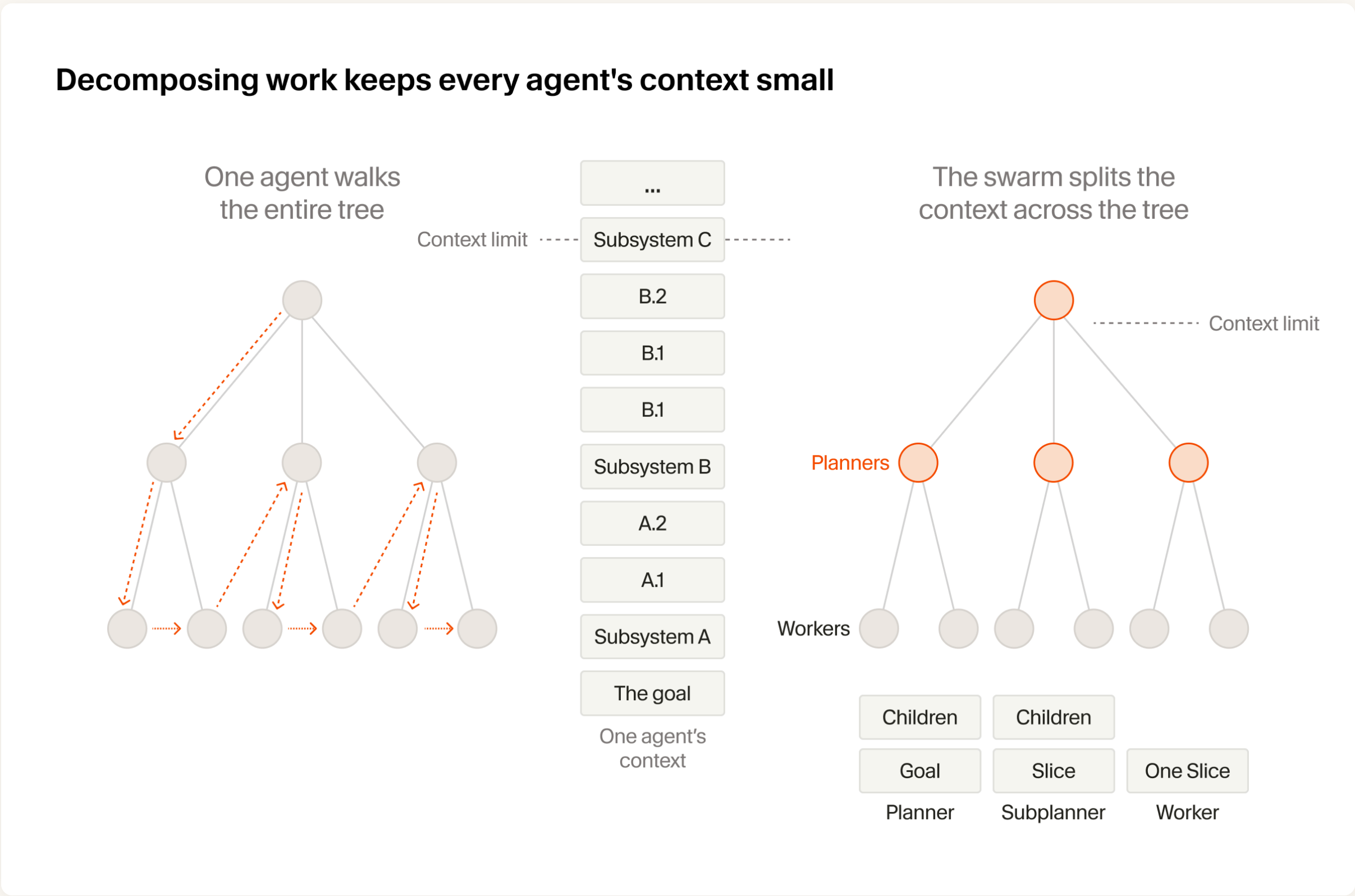
worker（橙）吃掉绝大部分 token，planner（灰）只是底下窄窄一条。但对照上面那张成本图看，这条窄的才是花钱的大头

worker 至少吃掉 69% 的 token，多数配置里超过 90%；但在 Opus + Composer 的组合里，作为 planner 的 Opus 只产生了很少一部分 token，却占了大约三分之二的成本。

Cursor 的结论是：**一个大任务里，真正需要前沿智能的时刻其实很少**——最初的拆解、设计决策、某些取舍，就这么几处。而一旦前沿模型把不确定性坍缩成了一条详细、明确的指令，后面的模型只要照着做就行了，用便宜的完全够。

它真正解决的其实是上下文效率

它自己给的解释我觉得比"并行"更准确：



左边是单 agent 走完整棵树，目标、子系统 A、B、C 一路往上堆，很快撞上下文上限；右边是 planner 只拿目标和一层 children，subplanner 拿一个 slice，worker 只拿一个 slice——每个人的上下文都很小

在蜂群里，planner 从来不写实现，所以它的上下文永远不会被底层细节填满；worker 从来不做规划，所以它能把全部上下文花在一小块活上。我们怀疑，蜂群之所以能上规模，靠的是这种上下文效率，而不是并行本身。

这句我觉得比"并行"这个词准确得多。

它还引了科斯的理论作类比：协调成本比工作本身增长得更快，所以组织会长成一层层有边界的单元，而不是让所有人跟所有人说话。

回到我最初的疑虑：冲突是真的会爆炸——两小时全库 7 万次，最热的那一个文件被 1,173 个 agent 轮番碰过。但它确实是可解的，靠的是仲裁 agent、自动拆文件、编译器可校验的决策引用，而不是靠模型变聪明。

第三章 · 怎么把它原生训进模型

● Kimi-把并行编排训进模型权重（PARL）

• PARL：只训 orchestrator，子 agent 冻住不动

K2.5 技术报告里这套训练框架叫 PARL（Parallel-Agent Reinforcement Learning），架构是解耦的：

PARL 框架采用了一个解耦的架构：一个可训练的 orchestrator，加上一批从固定的中间策略 checkpoint 实例化出来、权重冻结的子 agent。这个设计刻意避开了端到端联合优化，为的是绕开两个根本难题：功劳归属说不清，以及训练不稳定。

先说角色。orchestrator（编排器）是总指挥——它拿到你的任务，决定要不要拆、拆成几块、每块交给谁；子 agent 是干活的——各管自己那一小块，互相不说话，干完把结果交回来。

再说"从固定的中间策略 checkpoint 实例化"。模型训练时会像打游戏存档一样，每隔一段就把当前参数存一份，这份存档就叫 checkpoint。Kimi 的做法是：从训练途中挑一个存档，把它当成子 agent 的固定人格，之后再也不动它。所有子 agent 都是这个存档的副本，只是被塞了不同的岗位说明书。

最后是"权重冻结"。模型的本事都记在一堆参数（权重）里，训练就是不停改这些参数；冻结就是把子 agent 的参数锁死不许改，整场训练只有总指挥在变聪明。

用一个比方最好懂：**这是一支足球队，只练教练，不练球员。** 球员的水平从第一天到最后一天完全没变，变的只是教练怎么排兵布阵。

为什么这么做，论文讲得很清楚：多 agent 场景里，基于结果的奖励天生稀疏又嘈杂——最终答案对，不代表每个子 agent 都没出错；答案错，也不代表所有子 agent 都错了。所以他们干脆把子 agent 的产出当作"环境观测"，而不是可以求导的决策点——把"高层的协调逻辑"和"底层的执行能力"拆开各归各的，论文说这样训练收敛得更稳。

• serial collapse：模型会偷懒，退回单 agent

训一个可靠的并行 orchestrator 很难，因为子 agent 独立执行带来的反馈是延迟的、稀疏的、非平稳的。论文在这里同时防两种毛病：一种是不敢并行，一种是滥用并行。前一种有个专门的名字叫 serial collapse：

串行坍塌（serial collapse）——一种局部最优：orchestrator 默认退回到单 agent 执行。

"局部最优"这个词值得解释一下，因为它是理解后面那笔奖励的关键。

想象你在爬山，目标是最高峰。你现在站在一个小土包顶上：往哪个方向迈一步都是往下走，所以你不动了——但你离真正的山顶还差得远。这个小土包就是局部最优：周围都不如它，可它并不是最好的。

串行就是总指挥的那个小土包。一个人埋头干，虽然慢，但每一步都可控、结果也过得去；而"派 5 个人出去"这个动作在刚开始学的时候几乎必然翻车——派错人、拆错块、结果收不拢，分数当场变低。于是模型试了两次就学乖了：还是自己干吧。从此再也不碰并行，也就永远学不会并行。

模型不会自己一头扎进并行，它的默认选择是退回一个人干。所以得先用一笔额外的奖励，把它从这个舒适区里推出去——注意是"推出去探索"，不是"逼它永远并行"。论文说得很明确：并行本身不被预设为更优，到底要不要并行、什么时候并行、怎么拆，全都是靠环境反馈学出来的。

所以 PARL 给总指挥打的分是三个数加起来的：

$$r_{\text{PARL}}(x, y) = \lambda_1 \cdot r_{\text{parallel}} + \lambda_2 \cdot r_{\text{finish}} + r_{\text{perf}}(x, y)$$

- **r_perf**：任务层面的结果，评估解答的成功度和质量。
- **r_parallel**：实例化奖励，专门对抗 serial collapse——通过奖励"开子 agent"这个动作本身，逼它去探索并发调度的空间。
- **r_finish**：子 agent 完成率，防的是另一种作弊叫 spurious parallelism（虚假并行）——orchestrator 为了刷并行指标，疯狂开一堆没意义的子 agent。奖励"完成的子任务"就把这条路堵上了。

不用管符号，它读起来就是：总分 = 任务干得好不好 + $\lambda_1 \times$ 有没有真的并行 + $\lambda_2 \times$ 派出去的活有没有干完。前面那两个 λ 是权重，就是"这一项在总分里占多少斤两"的旋钮。

而最关键的一笔在这儿： λ_1 和 λ_2 这两个旋钮，在训练过程中被一路拧到 0。

"退火"是从炼钢那儿借来的词——金属加热之后要慢慢降温，不能一下冷透；放到训练里就是"这个数值随着训练一点点往下调，不是说切就切"。

翻译成带队的话：新手期，教练每换一次人、每成功派出去一个活，都单独给他记一功，逼他习惯用团队；等他用顺了，这些奖励一点点撤掉，最后只剩一条——赢没赢。

这一笔退火是整套设计的关键。如果并行奖励一直留着，模型最后一定会学成"为了拿分而拆任务"；撤掉之后，训练末期它没有任何理由再去并行了，可它还在并行——说明这时候它是真的算明白了并行更划算，而不是在领奖金。

• CriticalSteps：它优化的是关键路径

论文还定义了一个专门的计时单位，叫 critical steps（关键步数）。名字听着玄，讲的是一件生活常识：一件事由几拨人同时干，总耗时不等于所有人干活时间的总和，而是等于"每一轮里最慢那个人"的时间加起来。

就像装修房子：瓦工、木工、水电工同时开工，这一轮什么时候能结束，取决于最慢的那个工种，跟另外两个几点收工没关系。

$$\text{CriticalSteps} = \sum_{t=1}^T \left(S_{\text{main}}^{(t)} + \max_i S_{\text{sub},i}^{(t)} \right)$$

公式就是这个意思：把一次运行切成若干轮，每一轮 = 总指挥自己花的步数（论文说通常就是 1 步，也就是"派活"这个动作本身）+ 这一轮里跑得最久的那个子 agent 的步数；所有轮加起来，就是关键步数。

举个数：一轮里派了 10 个子 agent，9 个各跑 5 步就回来了，第 10 个跑了 40 步。这一轮记 41 步，不记 86 步——因为前 9 个等第 10 个的时候，时间并没有白流走。

反过来说，如果这 10 个人的活拆得均匀、每人 8 步，这一轮就只有 9 步。同样的总工作量，耗时差了四倍多，差别全在"最长那条腿有多长"。

这个指标很讲究：

如果只是拼命多开子任务，却没能缩短并行组里最长那条的执行时间，在这个指标下几乎没有收益；反过来，均衡的任务拆解只要缩短了最长的那条并行分支，就能直接降低 critical steps。

开一堆子 agent 但没缩短最长那条分支，在这个指标下几乎没有收益。所以 orchestrator 被引导去最小化端到端延迟，而不是最大化并发数或总工作量。

训练数据也是围着这个设计的：合成的 prompt 分两类，wide search（需要同时探索很多独立信息来源）和 deep search（需要多条推理分支、延迟聚合），另外掺了长文档分析和大规模文件下载这类真实负载。但论文特意强调了一句：

要强调的是，这些 prompt 并没有明确指示模型去并行。它们做的是塑造任务分布，让"并行拆解和调度"这种策略自然而然地占优。

不告诉模型要并行，而是把任务分布做成"并行才划算"的样子，让它自己学。

工程上还有一笔：为了提高效率，先用小尺寸的 subagent 训 orchestrator，之后再换成更大的模型。

• 它到底换来了什么

论文用三个 benchmark 验收，对照组是同一个 K2.5，只比"单 agent"和"开 swarm"：

- BrowseComp（深度检索）：60.6 → 78.4，涨 17.8 个点，超过了 GPT-5.2 Pro 的 77.9。
- WideSearch（广度检索，Item-F1）：72.7 → 79.0，涨 6.3 个点，越过 Claude Opus 4.5 的 76.2。
- 自建的 Swarm Bench：41.6 → 58.3，涨 16.7 个点。这套题包含开放网页检索、批量下载、100+ 份文档的广域阅读、10 万字以上的长文写作，是专门照着"值得并行"设计的，所以涨幅最大。

速度那一栏的口径要小心：论文说的 3—4.5 倍不是"总耗时快 3—4.5 倍"，而是"达到同一个质量目标所需的耗时之比"。在 WideSearch 上，目标 Item-F1 从 30% 抬到 70% 的过程中，单 agent 的耗时从约 1.8 倍一路涨到 7 倍以上，而 Agent Swarm 基本贴在 0.6—1.6 倍不动。换句话说：题越难，这个倍数越大；题很简单的时候，并行并不省时间。

附录里还有一个数很能说明问题：在 BrowseComp 上，orchestrator 自己的预算只有 15 步，而它派出去的每个子 agent 可以用 100 步。主脑几乎不干活，它的全部本事就在这十几步里——派谁、派几个、派去干什么。

• 专业化是涌现出来的

论文里有一张词云（Figure 6），图注写的是：这张词云展示的，是 orchestrator 在各项测试中动态实例化出来的、各不相同的 K2.5 子 agent。



论文 Figure 6：orchestrator 自己造出来的子 agent 角色名

里面字号最大的两个是 Biography Researcher（传记研究员）和 Verification Specialist（核实专员），紧跟着是 Award Researcher、Historical Researcher、Timeline Researcher、Cross Reference Analyst、University Researcher、Article Researcher 这一批；再往下越缩越小，一直到 fact checker、genealogy researcher、award investigator，最后是编号化的 verifier 2 / verifier 3、writer sec01 / sec03。

顺带说一句：这张图上的角色高度集中在"查人物、查奖项、查时间线、交叉核实"这一族，因为它主要来自 BrowseComp 这类深度检索题。所以它证明的是"角色由模型现场造"，而不是"模型什么职业都会造"。

这些角色没有一个是人写进角色表里的。这就是"无需预定义角色"最直观的证据。

如果编排本身可以被 RL 训进 orchestrator 的权重，那第二章里所有人在 harness 层写的调度代码、状态机、仲裁 agent，是不是过渡形态？

我倾向于短期内不是。PARL 训的是"要不要并行、怎么拆"这一层判断，而 Cursor 那 7 万次合并冲突、MiniMax 那三种成本，是执行环境本身的属性，不会因为 orchestrator 变聪明就消失。但最终agent的编排可能都会被训到基模里面。

第四章 · 怎么用它激发智能、找到真正的创新

到这儿为止，所有的努力都是在用 agent 的数量换速度。那能不能换智能？

第一类是"显然能做"的并行任务，也就是 Scaling 01 里那种：同时调研 50 个 benchmark、同时分析 500 个作者的文风、同时梳理 200 个 API 的入参出参。这类任务结构相同、互相独立、单个不难，天然适合 map-reduce。现在的 harness 基本都能做，剩下的差距在质量而不在能不能。

第二类是本质上不是并行、而是博弈的任务：一百个 agent 互相 judge、互相证伪、互相在对方的证明里找洞，最后收敛出一个单个 agent 到不了的结论。

我们来看看第二类是怎么做的

● Apodex (MiroMind) -同一个团队三代模型，一路把验证往外挪

这一节把三个模型放一起讲，因为它们是同一家的。

这家公司值得单独说两句。Apodex 的创始人是陈天桥。公司 2024 年成立，由他和清华大学电子工程系副教授代季峰一起筹办（代季峰已于 2026 年 1 月卸任），2025 年 8 月靠开源的 Miro Open Deep Research 第一次进入公众视野，总部在美国 Redwood City，2026 年 4 月 23 日改名 Apodex。

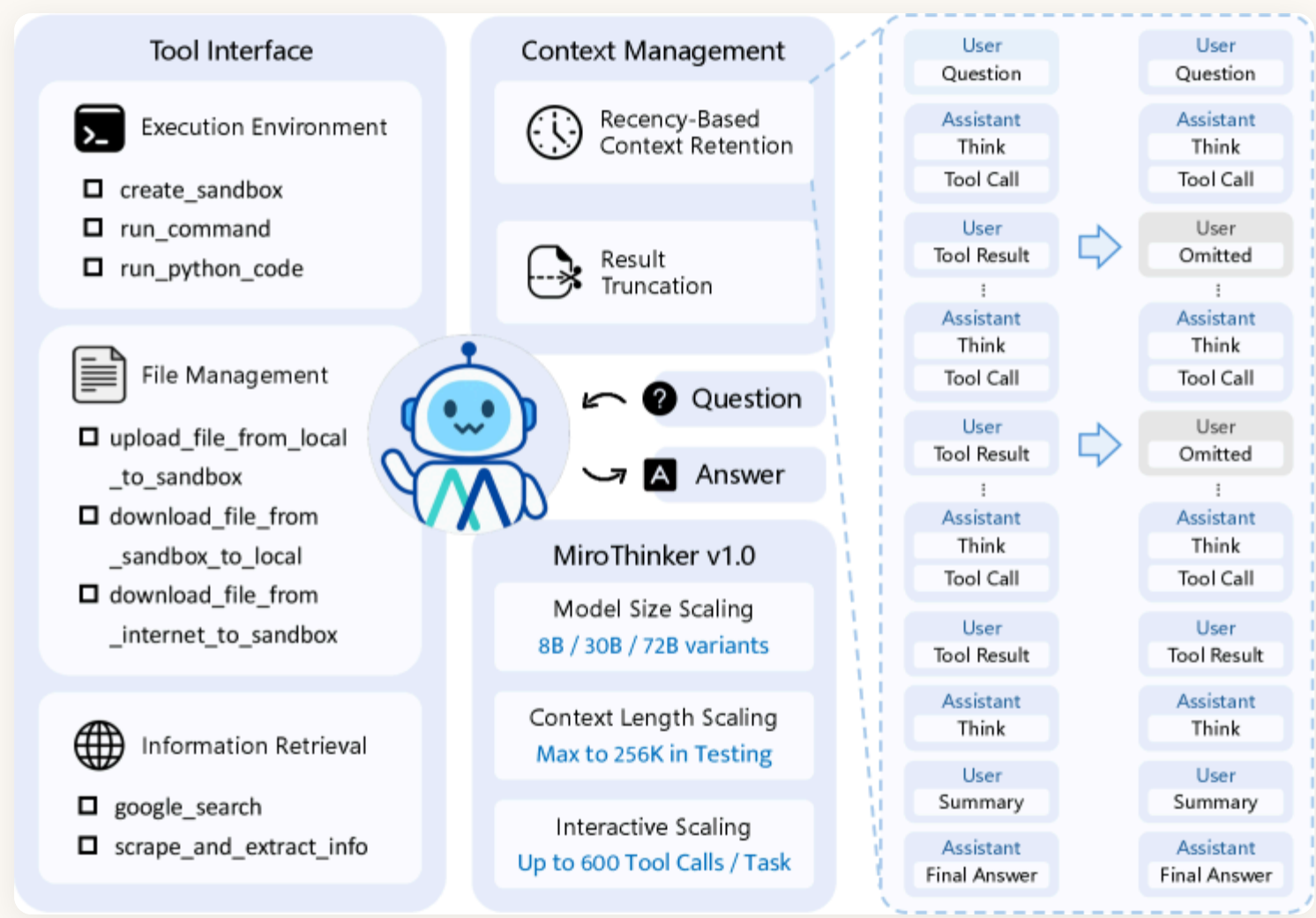
开源的 MiroThinker 系列和 MiroFlow 都出自这个团队。

它的愿景那句话，跟这篇文章的主线撞得很准：它还叫 MiroMind 的时候，官方对自己的定义是一个"可被证明是对的"（engineered to be provably right）的推理系统；改名之后换了个讲法，但意思没变——在前沿做研究、解题、发现，光有模型能力不够，得靠严谨的推理，而且每一步都被验过。产品上叫 "Heavy Duty Solver"，瞄的是那些难、赌注大、答错代价实打实的问题：官网列的是科研、金融、法律合规、临床推理。

所以它三代模型连起来看，主线特别清楚——从"把交互做深"，一路走到"把验证从生成里拆出去"。

• 第一代：交互深度是第三个 scaling 维度

MiroThinker v1.0 (arXiv:2511.11793) 提出 interaction scaling——在模型规模、上下文长度之外的第三个 scaling 维度。单任务最多 600 次工具调用、256K 上下文。72B 版本的最好成绩：GAIA 81.9%、HLE 37.7%、BrowseComp 47.1%、BrowseComp-ZH 55.6%——超过此前所有开源 agent，逼近 GPT-5-high 这类闭源产品。核心主张是交互深度本身就是一个可预测涨点的维度。



MiroThinker v1.0 的总览图。右下角那三行是它主张的三个 scaling 维度：模型大小（8B / 30B / 72B）、上下文长度（最高 256K）、以及交互次数（每个任务最多 600 次工具调用）——第三行才是这篇的新东西

• 第二代：把验证塞进推理过程本身

MiroThinker-1.7 & H1 (arXiv:2603.15726) 在这基础上加了两层验证：

Local Verifier 管的是"每一步走得对不对"。

它对付的毛病是：agent 每一步都会挑那条"看起来最顺"的路走。比如你让它查"某人 1998 年在哪家公司"，它第一反应是搜维基百科，搜到一条就收工了。局部验证就是在每一步之后追问一句：这一步真的推进了吗？还有没有别的方向没试？——别让"探索"退化成反复确认自己的第一直觉。

这一条的消融数据很硬：在 BrowseComp 的 hard 子集（295 题）上，加了 Local Verifier 之后准确率从 32.1 涨到 58.5，而交互步数从 1185.2 掉到 210.8。原文特意说了一句：步数减少不是设计目标，是局部验证的自然副产品。

Global Verifier 管的是"整条链撑不撑得住"。

它吃的是一个不对称性：让你从头写一个证明很难，但让你检查一个证明对不对，容易得多。所以它把收集到的证据整条摊开看一遍——如果 agent 说"答案是 A"，但支持 A 的只有一条二手转述，那就不收，打回去让它重新采样、或者把证据链补全，而不是硬着头皮交一个答案。如果手上同时有好几条候选路径，它还会横着比一遍，在给定的算力预算内挑出证据链最完整、最经得起查的那一条——不是挑说的人最多的那一条。这一步就是下一代那套"看整体证据而不是看票数"的雏形。

• 第三代：heavy-duty 模式，和一个可以说不的 global verifier

先解释一下 "heavy-duty"（大致可以译成"重载"）。它不是另一个模型，是同一个模型的另一档运行模式——就像同一辆车挂低速四驱：慢、费油，但能爬平时上不去的坡。普通模式是你问一句它答一句；heavy-duty 是你把一个真研究问题丢进去，它可能花上几个小时、开出上百个 agent、走上万步，最后交一份带引用的报告回来。你会为一条微博开这个模式吗？不会。你会为一份要拿去做投资决策的尽调开吗？会。

到了 Apodex-1.0，验证不再只是推理过程里的一环，而是被抬成了整个架构的中心。

Apodex-1.0（官方技术博客，原文把这套东西描述成"一支异步的 agent 团队：各自专精、互相交叉核对、在回答之前先审一遍自己的证据"）的 heavy-duty 模式结构最完整。原文把整条链路写成了一段话，我拆开讲：

- 1. 拆：一个总调度（orchestrator）接到问题，拆成一堆子任务。

2. 派：每个子任务派一个专门的 agent，各自带自己的上下文、指令和工具——查文献的和跑数据的，工具箱是不一样的。
3. 收：所有 agent 干完把报告丢进一个共享的报告池。这里有个细节值得停一下：总调度不等，它只是隔一会儿去瞄一眼池子边上那张"谁交了、谁还没交"的状态表。这就意味着一个卡住的 agent 拖不垮整场——这正是第二章 MiniMax 那个"被最慢的 worker 卡死"的正面答案。
4. 验：全部收齐之后，才轮到最后那个 global verifier 出场，它对着拼起来的全部证据从头推一遍，然后才允许输出答案。

单个任务里，这套东西最多能协调 150 个子 agent、跑超过 15,000 步，官方说法是"比单 agent loop 的饱和点高两个数量级"——翻译一下：一个单 agent 跑到一两百步就开始原地打转、再跑也不涨了，而这套东西能一路跑到 15,000 步还在往前推。作为对照，同一家第一代 MiroThinker 的上限是 600 步。

关键差别在于：**最后拍板的那个 agent，不在干活的那批里面。**

原文的说法是，verifier 拿到的指令是"评估"，不是"接着往下推"，而且它有权说不同意。这跟"让干活的人自己复查一遍"是两回事。

而且它不是投票。原文把这个区别讲得很清楚：问题从"哪个答案得票最多"换成了"整体证据支持什么"——三个 agent 都说 A，但三个人的依据都来自同一篇不靠谱的文章，那 A 照样不成立。

中间还有一支专门的验证小队随时待命：两份报告打架，派 conflict reviewer；某条结论需要落实，派 fact checker；草稿要过最后一遍，派 draft-report reviewer。

同一个 Apodex-1.0，开不开 heavy-duty 差多少？BrowseComp 上从 75.5 涨到 90.3 (+14.8)，FrontierScience-Research 上从 28.3 涨到 46.7 (+18.4) ——越是没有现成答案、越要自己拼证据的题，这套外置验证的收益越大。

还有个反直觉的点：同一个 Apodex-1.0，开了 heavy-duty 之后总步数经常比不开的时候还少——因为 verifier 会把那些不产生信息增益的步骤滤掉，把算力集中到真正推进问题的地方。

这跟 Cursor 那边"少写四分之三代码把同一件事做对"是同一个现象：组织方式对了，效率和质量是一起涨的，不是拿一个换另一个。

Evomap 那 166 个丢掉的答案、Cursor 的中立仲裁 agent、MiniMax 的 Verifier、Claude Code **/deep-research** 里的投票过滤、Apodex 的 global verifier——都是同一件事：不能让生成者自己当裁判。

● DeepMind AlphaProof Nexus-集群买的是能力，还是成本

5 月 DeepMind 的 AlphaProof Nexus (the-decoder 报道、arXiv:2605.22763) 把这条路走通了一段：

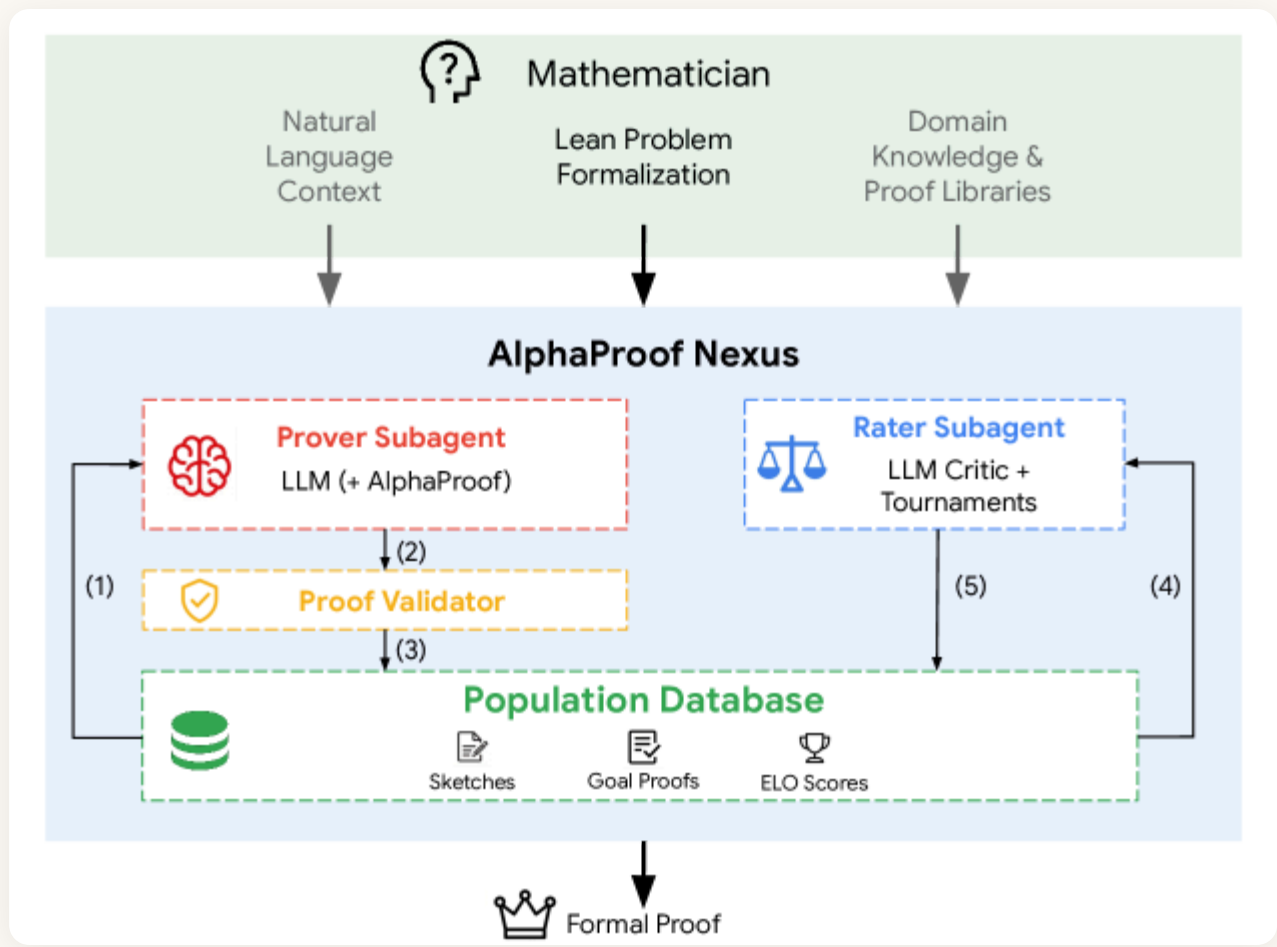
- 在 353 个尝试的 Erdoš 问题里解出 9 个，其中两个悬了 56 年。
- 证明了 OEIS 上 492 个开放猜想里的 44 个（这 44 个是经人工复核确认"形式化正确且此前未被证明"的净值）。

- 解决了代数几何里一个悬了 15 年的开放问题——证明了 codimension 3、type 2 这一档 pure O-sequences 的 log-concavity（注意是这个大猜想里的一个特例，不是整个猜想）；还在凸优化里一边找证明一边发现了一个新的参数调度，把一个此前开放的收敛率问题做到了精确的 $O(1/t)$ 。
- 每个问题的推理成本"几百美元"。

它一共试了四档配置，可以理解成给同一个 agent 逐级加装备：

- A 档（最朴素）：一池子 Gemini 3.1 Pro 各写各的证明，互相不通信、不共享任何中间结果；每个都是写完丢给 Lean 编译器检查，编译不过就把报错喂回去重写，如此死循环。注意它本来就已经是并行的了——它缺的不是"多"，是"这些 agent 之间有没有关系"。
- B 档：在 A 上面加外援——卡壳的时候可以调 AlphaProof，那是 DeepMind 用强化学习专门训出来的定理证明器，擅长把证明里缺的那几步补上。
- C 档：换个方向加——让一群子 agent 同时写证明草图，草图全都丢进一个共享池子；另有一批评分 agent 两两比较这些草图谁更有戏，汇总成 Elo 分（就是国际象棋排名用的那套分）。分高的草图被优先拿来继续往下演化。说白了，就是给证明思路搞了个赛马。
- D 档：B 和 C 全都要。

那 9 道题是 D 档跑出来的。



AlphaProof Nexus 的架构。左边红框是写证明的 Prover Subagent（LLM，可选调 AlphaProof），产出先过黄框的 Proof Validator，再存进绿框那个共享的 Population Database（草图、目标证明、ELO 分）；右边蓝框是打分用的 Rater Subagent，靠"锦标赛"式的两两比较给草图排名，排名再反过来指导下一轮怎么写。最上面那位数学家负责把问题形式化成 Lean

总结

这半年最直观的变化是数量——从"3 到 5 个 teammate"到"一次派 1000 个"，只用了三个月。但把六家的做法摆在一起看，**难的从来不是拆，是合**：Evomap 那 166 个算对了却没送到的答案、Cursor 那 7 万次合并冲突、MiniMax 把"汇总"单列成一种成本，讲的都是同一件事。

各家的解法也殊途同归——**能用确定性程序做的调度和汇总，就别交给 LLM**。MiniMax 把它写成状态机，Anthropic 把它写成脚本，Cursor 派了个中立的仲裁 agent；Kimi 把编排能力直接 RL 进了模型权重，而且发现模型并不会自发学会并行，得专门设一笔奖励，把它从"还是自己干吧"的舒适区里推出去。

至于用 agent 数量换智能、而不只是换速度，目前唯一靠得住的东西是**一个不会被说服的裁判**：Lean 编译器、sqllogictest、外置的 global verifier。有这么个东西在链条末端，前面开多少个 agent 都不太会失控；没有的话，开得越多，错得越齐。

麻烦的是"哪个点子更好"这类问题恰恰最难找到裁判。你没法编译一个创意，也没有哪个测试套件能告诉你哪个角度更有价值。但真正的AI4SI往往藏在那1000次rollout的某次中。